thws Technische Hochschule Würzburg-Schweinfurt

Projektarbeit

INTERAKTIVE LERNPLATTFORM FÜR GRUNDLAGEN INFORMATIK

M. Nöth, R. Schröder, E. Ebert, D. T. Pham, K. Popov

1. KONTEXT UND ZIELBILD

AUSGANGSLAGE

- Themen wie Automaten, Zahlensysteme und Kodierungen sind abstrakt.
- Statische Aufgaben zeigen meist nur Eingabe und Ergebnis.
- Lernende sollen Zwischenschritte selbst verändern und beobachten können.

ZIEL DER PROJEKTARBEIT

- Eine browserbasierte Lern- und Übungsplattform für *Grundlagen Informatik*.
- Interaktive Simulatoren erklären den Rechen- oder Ausführungsweg Schritt für Schritt.
- Einheitliche Oberfläche für zwei Lernbereiche und acht Fachthemen.

2. ANFORDERUNGEN UND PROJEKTABLAUF

ANFORDERUNGEN ALS LEITPLANKEN

- Didaktik: nicht nur Ergebnisse, sondern Zwischenschritte sichtbar machen.
- Bedienung: gleiche Navigation, Tabs und Rückmeldungen in allen Themen.
- Technik: neue Themen sollen ohne Änderungen an Navigation und Routing ergänzt werden können.
- Qualität: Deutsch und Englisch, helles und dunkles Erscheinungsbild sowie automatisierte Prüfungen.

ANFORDERUNGSPRÜFUNG MIT LEHRENDEN

- **Hier den tatsächlichen Nachweis ergänzen:** Gesprächspartner, Datum und Anlass.
- Beispielhafte Folienstruktur: Anforderung aus dem Gespräch → Umsetzung → beobachtetes Ergebnis.
- Geeignete Belege: Gesprächsnotiz, freigegebene E-Mail, Protokoll oder Feedback zu einem Prototypen.
- Ohne diesen Beleg sollte die Präsentation keine Kundengespräche behaupten.

PROJEKTABLAUF

- Anforderungen und Themenumfang aus Lehrmaterial und Lastenheft ableiten.
- Architektur und gemeinsame UI-Bausteine vor den Fachmodulen festlegen.
- Fachmodule in Feature-Branches entwickeln und per Merge Request integrieren.
- Tests, Build und Lighthouse-Prüfung in GitLab ausführen; stabile Stände nach `main` überführen.

3. LÖSUNGSANSATZ: ARCHITEKTUR UND DESIGN

TECHNISCHE BASIS

- React 19 und TypeScript bilden die Anwendungslogik.
- Vite erzeugt das auslieferbare Frontend; Tailwind CSS und shadcn/ui gestalten die Oberfläche.
- Zustand hält wenige globale UI-Zustände, etwa Theme und Seitenleiste.
- i18next liefert Texte in Deutsch und Englisch.

MODULARE ARCHITEKTUR

- `moduleCatalog.tsx` registriert Module, Themen, Routen und Navigation an einer Stelle.
- Gemeinsame Seitengerüste liegen in `components/shared`; fachlicher Inhalt bleibt im jeweiligen Themenordner.
- Ganze Seiten werden lazy geladen. Dadurch bleibt die erste Auslieferung kleiner.
- Die Trennung verhindert doppelte Registrierung in Router, Header, Sidebar und Modulübersicht.

WIEDERVERWENDBARES LERNMUSTER

- Jedes Thema verwendet dieselben Bereiche: Überblick, Übungen, Beispiele und Simulation.
- `ModuleTopicPage` steuert Tabs, Aufgabenwahl und den Wechsel zur Simulation.
- Themenordner binden nur ihre konkreten Inhalte und Simulatoren ein.
- Ergebnis: ein neues Thema folgt einer dokumentierten Ordner- und Registrierungskonvention.

DESIGNENTSCHEIDUNGEN FÜR SIMULATOREN

- Konfiguration und Ablaufzustand sind getrennt; die Simulation bleibt reproduzierbar.
- Schrittsteuerung macht Zustände und Übergänge nachvollziehbar.
- Fachmodule nutzen eigene Visualisierungen, etwa Zustandsdiagramme, Wahrheitstabellen oder Bitfolgen.
- Import und Export erlauben, einen Arbeitsstand weiterzugeben oder später fortzusetzen.

CODE-ANKER FÜR DIE BEFRAGUNG

- `data/moduleCatalog.tsx`: zentrale Registrierung von Modulen, Routen und Navigation.
- `components/shared/ModuleTopicPage/ModuleTopicPage.tsx`: Tabs, Aufgabenwahl und Wechsel zur Simulation.
- `hooks/useSimulationEngine.ts` und `SimulationWorkspace.tsx`: Ablaufzustand und gemeinsame Simulator-Oberfläche.
- Zusätzlich zeigt jedes Teammitglied ein verantwortetes Fachmodul mit einem konkreten Testfall.

FACHLICHER UMFANG

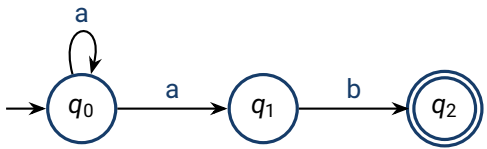
- Grundlagen Informatik: Turing-Maschinen, endliche Automaten, reguläre Sprachen, Zahlensysteme, Moore/Mealy, DNF und Kodierungen.
- Programmierung 1: Java-Programmierung mit Editor und Ausführung im Browser.
- Gemeinsames Ziel aller Module: Eingabe bearbeiten, Ergebnis prüfen und den Lösungsweg sichtbar machen.

FACHMODUL: ENDLICHE AUTOMATEN

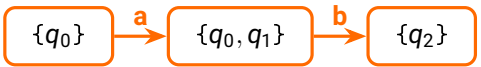
ENDLICHE AUTOMATEN – ÜBERBLICK

- Studierende definieren **DFA und NFA** selbst und führen ein Eingabewort Schritt für Schritt aus.
- Jeder Schritt erscheint gleichzeitig in **Übergangstabelle** und **Zustandsdiagramm**.
- Ein **Stapeltest** prüft mehrere Wörter auf einmal – erkennt der Automat die beabsichtigte Sprache?
- Ein Automat lässt sich aus einer **rechtslinearen Grammatik** erzeugen.
- Übungen geben eine automatische Rückmeldung mit Gegenbeispiel.

EIN AUSFÜHRUNGSKERN FÜR DFA UND NFA



Nichtdeterminismus:
a führt in zwei Zustände



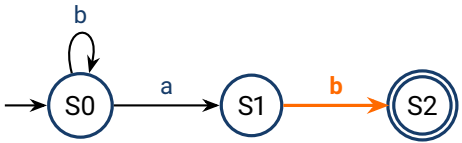
beim DFA enthält die Menge höchstens ein Element

Der Kern führt immer eine **Zustandsmenge** aus: $S' = \varepsilon\text{-Hülle}\left(\bigcup_{q \in S} \delta(q, a)\right)$
Der deklarierte Typ (DFA/NFA) steuert nur die **Validierung**, nicht den Ausführungspfad.

UI: ZWEI SICHTEN, DERSELBE SCHRITT

δ	$\rightarrow$S0	S1	*S2
a	S1	S1	S1
b	S0	**S2**	S0

Übergangstabelle: die formale Definition



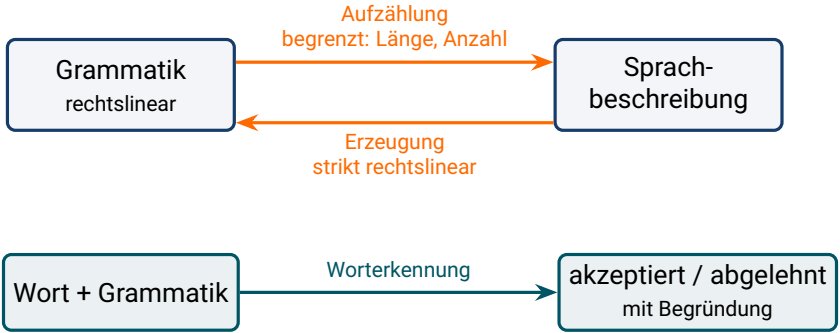
Zustandsdiagramm: der Weg des Wortes

markierte **Zelle** und markierte **Kante** stammen aus demselben Simulationsschritt

FACHMODUL: REGULÄRE SPRACHEN

thws
Technische Hochschule
Würzburg-Schweinfurt

REGULÄRE SPRACHEN – DREI ARBEITSWEISEN



Alle drei Modi bleiben im Hintergrund erhalten und werden nur ein- oder ausgeblendet – dadurch überlebt eine begonnene Eingabe den Moduswechsel.

BEWUSST ABGELEHNTE SPRACHBESCHREIBUNGEN

```
L = {a^n b^n | n > 0}
```

nicht regulär – dieselbe Variable in zwei Faktoren koppelt die Wortlängen aneinander

```
L = {a^n | n < 5}
```

obere Schranke – beschreibt zwar eine endliche Sprache, ist als Aufzählung aber klarer

Beide Eingaben ließen sich formal verarbeiten. Sie werden trotzdem mit einer **eigenen Meldung** zurückgewiesen: Diese Fälle ausdrücklich zu erklären ist lehrreicher, als sie stillschweigend zu verwerfen.

ÜBUNGSPRÜFUNG: VERHALTEN STATT STRUKTUR

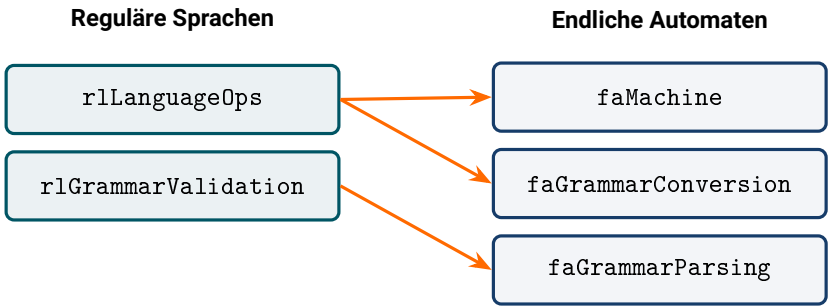


gleiche Entscheidung für jedes Wort?

ja ⇒ bestanden
nein ⇒ erstes abweichendes Wort als Gegenbeispiel

Ein **Strukturvergleich** würde korrekte Lösungen ablehnen, die nur anders aufgebaut sind. Die begrenzte Wortmenge ist bewusst eine **didaktische Rückmeldung** und kein formaler Äquivalenznachweis. Strukturelle Mängel werden vorrangig gemeldet – sie sind meist die eigentliche Ursache.

WIEDERVERWENDUNG STATT DUPLIKAT



Abhängigkeit nur in eine Richtung – kein Zyklus

Das Verzeichnis `domain` hat **keine Abhängigkeit zu React** und bleibt ohne gerenderte Oberfläche testbar. Die Aufzählung führt eine Grammatik über denselben Automatenkern aus wie die Simulation – eine eigene Ableitungslogik hätte zwei Implementierungen desselben Verhaltens erfordert.

4. KI-EINSATZ IM PROJEKT

KI ALS ENTWICKLUNGSWERKZEUG

- Genutzt wurden ChatGPT, Codex, lokal ausgeführte Modelle über Ollama, Gemini und Claude (Opus).
- Einsatzfelder: Strukturierung von Arbeitspaketen, Entwürfe, Refactorings, Tests und Varianten von Übungsaufgaben.
- KI-Ausgaben waren Vorschläge. Fachliche Verantwortung und Freigabe blieben im Team.

TOOLING: LSP, MCP & GRAPHEN

- **LSP-Integration** (`typescript-lsp`): konsistente Typen und sichere Refactorings über Dateigrenzen.
- **Browser-MCP** (`chrome-devtools-mcp`): Debugging von Simulator-Zuständen direkt im Browser.
- **Graphify-Wissensgraph**: Semantischer Projektkontext verhindert Kontextverlust bei großen Änderungen.
- **Aufgabengenerierung**: KI erzeugt Varianten aus Vorlagen; Verifikation erfolgt deterministisch im Simulator.

ARBEITSWEISE MIT KI

- Aufgabe und bestehende Architektur zuerst präzise beschreiben.
- Vorschlag gegen Fachmodell, Codebasis und Projektkonventionen prüfen.
- Kleine, testbare Änderungen übernehmen; anschließend Review und automatisierte Prüfungen ausführen.
- Bei Aufgabenmaterial Lösungen unabhängig nachvollziehen.

GRENZEN UND ERKENNTNISSE

- Große Gesamtaufgaben verlieren Details oder erzeugen unnötige Abstraktionen und Codeduplikate.
- Bei Automaten und formaler Logik waren Format und fachliche Korrektheit nicht verlässlich genug für eine ungeprüfte Übernahme.
- Der Nutzen entstand durch klare Teilaufgaben, Tool-Unterstützung und Kontrolle – nicht durch automatisches Generieren.

5. QUALITÄT UND DEPLOYMENT-STRATEGIE

QUALITÄTSSICHERUNG

- GitLab-Pipeline prüft Abhängigkeitssignaturen, Formatierung und Linting.
- Vitest prüft Unit-Tests; Playwright prüft zentrale Abläufe im Browser.
- Der Build erzeugt das Frontend-Artefakt; Lighthouse prüft die ausgelieferte Anwendung.
- Ein GitLab Runner auf einem Raspberry Pi führt die ARM-basierten Container-Jobs aus.

ABGLEICH MIT DEM LASTENHEFT

- **Fachlicher Umfang**: Alle Lastenheft-Ziele erfüllt und erweitert (Reguläre Sprachen, Kodierungen, Programmieren 1).
- **Didaktischer Kern**: Schrittweise Simulationen, sichtbare Zwischenschritte und begründete Rückmeldungen ohne Backend.
- **Messbare Performance**: Automatisiertes Lighthouse CI-Gate in der Pipeline ($LCP \leq 1{,}25\,\text{s}$) über alle Themen- und Einstiegsseiten.
- **Offene Punkte**: Usability-Tests mit Studierenden, Screenreader-Barrierefreiheit und mobile Optimierung.

DEPLOYMENT-STRATEGIE

- Die Anwendung ist ein Vite-Frontend. Der Build erzeugt statische Dateien in `app/dist`.
- Nach erfolgreicher Pipeline ist dieses Artefakt der auslieferbare Stand.
- Für eine Produktivsetzung fehlen im Repository noch Zielsystem, Freigabeschritt und Rollback-Ablauf.
- Für die Demo nutzen wir einen vorab geprüften Build und einen zweiten Browser als Rückfallebene.

6. INTERAKTIVE LIVE-DEMO

ABLAUF DER 45 MINUTEN

- Kontext und Anforderungen: 10 Minuten
- Architektur, Design und KI-Einsatz: 15 Minuten
- Qualität und Deployment: 5 Minuten
- Interaktive Live-Demo: 15 Minuten

AUFGABE AN DIE PRÜFENDEN

Öffnen Sie ein Fachthema Ihrer Wahl. Wählen Sie eine Übung oder ein Beispiel, verändern Sie eine Eingabe und starten Sie die Simulation. Beschreiben Sie anschließend, welcher Zwischenschritt die Lösung erklärt.

MODERATION DER DEMO

- Zwei Endgeräte oder zwei Browserfenster vorbereiten: Prüfende bedienen eines selbst.
- Aufgabe A: Wort in einem endlichen Automaten ändern und die Zustandsfolge prüfen.
- Aufgabe B: DNF- oder Zahlensystem-Aufgabe bearbeiten und Rückmeldung auslösen.
- Wir zeigen danach Import/Export eines Arbeitsstands und ordnen die Ansicht der Architektur zu.

DEMO-ABSICHERUNG

- Startzustand und direkte URLs vor der Präsentation testen.
- Eine vorbereitete Eingabe und ein gespeicherter Arbeitsstand liegen lokal bereit.
- Fällt die Bedienung durch die Prüfenden aus, übernimmt ein Teammitglied denselben Ablauf.

7. AUSBLICK UND FAZIT

ERWEITERBARKEIT UND PERSPEKTIVEN

- **Lernbereiche**: Neue Module lassen sich zentral im `moduleCatalog` ohne Umbauten registrieren.
- **Aufgabenverwaltung**: Zukünftige Erstellung und Variantenbildung von Aufgaben direkt über die Oberfläche.
- **Lernunterstützung**: PDF-Arbeitsblattexport für Präsenzveranstaltungen und Theorie-Quizze.
- **Infrastruktur**: Abwägung zwischen bewusster Offline-/Backendfreiheit und optionaler LMS-/Moodle-Anbindung.

NÄCHSTE SINNVOLLE SCHRITTE

- Echtes Feedback aus Lehrveranstaltungen dokumentieren und in Anforderungen zurückführen.
- Produktiv-Deployment mit Hosting, Freigabe und Rückfallplan festlegen.
- Mobile Optimierung für komplexe Tabellen und Zustandsdiagramme auf kleinen Bildschirmen.

FAZIT

- Die Plattform verbindet fachliche Modelle mit bearbeitbaren, schrittweisen Simulationen.
- Architektur und gemeinsame UI-Bausteine halten die Fachmodule konsistent und erweiterbar.
- Die Live-Demo prüft den Kernanspruch unmittelbar: Die Plattform erklärt sich durch Benutzung.